



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 05/03/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Instrucciones:

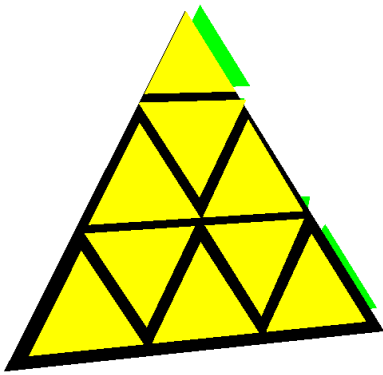
1.- Generar una pirámide rubik (pyraminx) de 9 pirámides por cara.
Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña)

Agregar en su documento escrito las capturas de pantalla necesarias para que se vean las 4 caras de toda la pyraminx o un video en el cual muestra las 4 caras

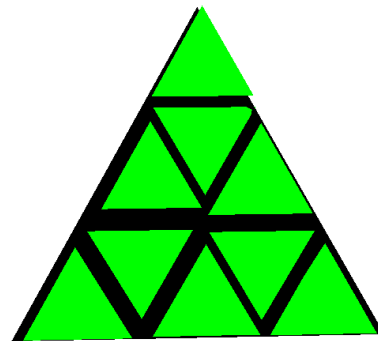
Capturas de pantalla del funcionamiento Actividad 1:

práctica

Practica XX: Nombre de la práctica



Cara amarilla



Cara verde

Código generado:

Modificación formación de pirámide para vértices a la misma distancia:

```
// Pirámide triangular regular
void CrearPiramideTriangular()
{
    // Índices para las 4 caras triangulares
    unsigned int indices_piramide_triangular[] = {
        0, 1, 2, // Cara 1
        1, 3, 2, // Cara 2
        3, 0, 2, // Cara 3
        1, 0, 3  // Base
    };

    // Vértices de un tetraedro regular centrados en el origen
    // La distancia entre todos los vértices es aproximadamente 1.0
    GLfloat vertices_piramide_triangular[] = {
        -0.5f, -0.288f,  0.288f,  // 0: Frente Izquierda
        0.5f, -0.288f,  0.288f,  // 1: Frente Derecha
        0.0f,  0.528f,  0.0f,    // 2: Cúspide (Altura ajustada)
        0.0f, -0.288f, -0.577f   // 3: Atrás Centro
    };

    Mesh* obj1 = new Mesh();
    // Nota: Usamos 12 valores de floats y 12 índices
    obj1->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
    meshList.push_back(obj1);
}
```

Creación de la pirámide negra:

```
343 //piramide negra (cuerpo)
344 model = glm::mat4(1.0);
345 model = glm::translate(model, glm::vec3(0.0f,0.0f,-4.0f));
346 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
347 color = glm::vec3(0.0f, 0.0f, 0.0f);
348 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
349 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
350 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
351 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
352 meshList[1]->RenderMeshGeometry();
353
```

Código cara amarilla:

```
355 //cara amarilla
356
357 //pir der inf
358 model = glm::mat4(1.0);
359 model = glm::translate(model, glm::vec3(-0.3f, -0.17f, -3.79f));
360 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
361 color = glm::vec3(1.0f, 1.0f, 0.0f);
362 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
363 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
364 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
365 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
366 meshList[1]->RenderMeshGeometry();
367
368 //pir izq inf
369 model = glm::mat4(1.0);
370 model = glm::translate(model, glm::vec3(0.3f, -0.17f, -3.79f));
371 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
372 color = glm::vec3(1.0f, 1.0f, 0.0f);
373 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
374 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
375 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
376 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
377 meshList[1]->RenderMeshGeometry();
378
379 //pir med inf
380 model = glm::mat4(1.0);
381 model = glm::translate(model, glm::vec3(0.0f, -0.17f, -3.79f));
382 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
383 color = glm::vec3(1.0f, 1.0f, 0.0f);
384 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
385 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
386 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
387 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
388 meshList[1]->RenderMeshGeometry();
389
390 //pir der inv inf
391 model = glm::mat4(1.0);
392 model = glm::translate(model, glm::vec3(-0.15f, -0.10f, -3.82f));
393 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
394 color = glm::vec3(1.0f, 1.0f, 0.0f);
395 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
396 model = glm::rotate(model, glm::radians(40.0f), glm::vec3(1.0f, 0.0f, 0.0f));
397 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
398 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
399 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
400 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
401 meshList[1]->RenderMeshGeometry();
402
```

```

403 //pir izq inv der
404 model = glm::mat4(1.0);
405 model = glm::translate(model, glm::vec3(0.15f, -0.10f, -3.82f));
406 model = glm::scale(model, glm::vec3(0.27f, 0.28f, 0.28f));
407 color = glm::vec3(1.0f, 1.0f, 0.0f);
408 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
409 model = glm::rotate(model, glm::radians(40.0f), glm::vec3(1.0f, 0.0f, 0.0f));
410 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
411 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
412 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
413 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
414 meshList[1]->RenderMeshGeometry();
415
416 //pir der sup
417 model = glm::mat4(1.0);
418 model = glm::translate(model, glm::vec3(0.14f, 0.12f, -3.88f));
419 model = glm::scale(model, glm::vec3(0.27f, 0.28f, 0.28f));
420 color = glm::vec3(1.0f, 1.0f, 0.0f);
421 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
422 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
423 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
424 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
425 meshList[1]->RenderMeshGeometry();

```

```

426
427 //pir izq sup
428 model = glm::mat4(1.0);
429 model = glm::translate(model, glm::vec3(-0.14f, 0.12f, -3.88f));
430 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
431 color = glm::vec3(1.0f, 1.0f, 0.0f);
432 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
433 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
434 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
435 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
436 meshList[1]->RenderMeshGeometry();
437
438 //pir med inv sup
439 model = glm::mat4(1.0);
440 model = glm::translate(model, glm::vec3(0.0f, 0.2f, -3.91f));
441 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
442 color = glm::vec3(1.0f, 1.0f, 0.0f);
443 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
444 model = glm::rotate(model, glm::radians(40.0f), glm::vec3(1.0f, 0.0f, 0.0f));
445 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
446 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
447 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
448 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
449 meshList[1]->RenderMeshGeometry();
450

```

```

451 //pir sup
452 model = glm::mat4(1.0);
453 model = glm::translate(model, glm::vec3(0.0f, 0.4f, -3.995f));
454 model = glm::scale(model, glm::vec3(0.24f, 0.24f, 0.24f));
455 color = glm::vec3(1.0f, 1.0f, 0.0f);
456 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
457 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
458 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
459 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
460 meshList[1]->RenderMeshGeometry();
461

```

Código cara verde:

```
463 //cara verde
464
465 //pir inf izq
466 model = glm::mat4(1.0);
467 model = glm::translate(model, glm::vec3(0.37f, -0.17f, -3.85f));
468 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
469 color = glm::vec3(0.0f, 1.0f, 0.0f);
470 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
471 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
472 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
473 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
474 meshList[1]->RenderMeshGeometry();
475
476
477 //pir inf der
478 model = glm::mat4(1.0);
479 model = glm::translate(model, glm::vec3(0.05f, -0.17f, -4.4f));
480 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
481 color = glm::vec3(0.0f, 1.0f, 0.0f);
482 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
483 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
484 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
485 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
486 meshList[1]->RenderMeshGeometry();
487
488
489 //pir inf med
490 model = glm::mat4(1.0);
491 model = glm::translate(model, glm::vec3(0.20f, -0.17f, -4.135f));
492 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
493 color = glm::vec3(0.0f, 1.0f, 0.0f);
494 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
495 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
496 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
497 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
498 meshList[1]->RenderMeshGeometry();
499
500 //pir inf inv izq
501 model = glm::mat4(1.0);
502 model = glm::translate(model, glm::vec3(0.3f, -0.1f, -3.995f));
503 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
504 color = glm::vec3(0.0f, 1.0f, 0.0f);
505 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
506 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
507 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
508 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
509 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
510 meshList[1]->RenderMeshGeometry();
```

```

511 //pir inf inv der
512 model = glm::mat4(1.0);
513 model = glm::translate(model, glm::vec3(0.15f, -0.1f, -4.28f));
514 model = glm::scale(model, glm::vec3(0.28f, 0.28f, 0.28f));
515 color = glm::vec3(0.0f, 1.0f, 0.0f);
516 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
517 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
518 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
519 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
520 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
521 meshList[1]->RenderMeshGeometry();
522
523 //pir sup izq
524 model = glm::mat4(1.0);
525 model = glm::translate(model, glm::vec3(0.11f, 0.10f, -3.92f));
526 model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
527 color = glm::vec3(0.0f, 1.0f, 0.0f);
528 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
529 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
530 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
531 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
532 meshList[1]->RenderMeshGeometry();
533

```

```

534 //pir sup der
535 model = glm::mat4(1.0);
536 model = glm::translate(model, glm::vec3(0.03f, 0.12f, -4.2f));
537 model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
538 color = glm::vec3(0.0f, 1.0f, 0.0f);
539 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
540 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
541 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
542 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
543 meshList[1]->RenderMeshGeometry();
544
545
546 //pir sup inv med
547 model = glm::mat4(1.0);
548 model = glm::translate(model, glm::vec3(0.1f, 0.2f, -4.08f));
549 model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
550 color = glm::vec3(0.0f, 1.0f, 0.0f);
551 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
552 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
553 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
554 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
555 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
556 meshList[1]->RenderMeshGeometry();
557

```

```

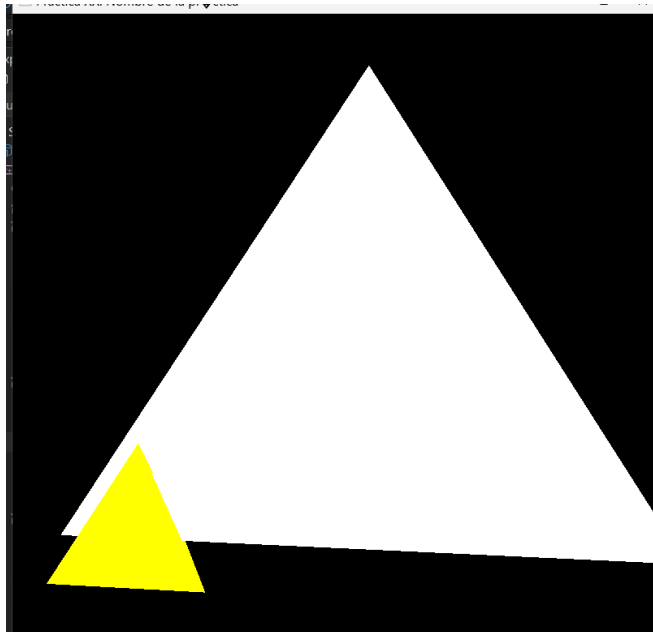
558 //pir sup inv med
559 model = glm::mat4(1.0);
560 model = glm::translate(model, glm::vec3(0.05f, 0.41f, -4.04f));
561 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
562 color = glm::vec3(0.0f, 1.0f, 0.0f);
563 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
564 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
565 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
566 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
567 meshList[1]->RenderMeshGeometry();
568
569

```

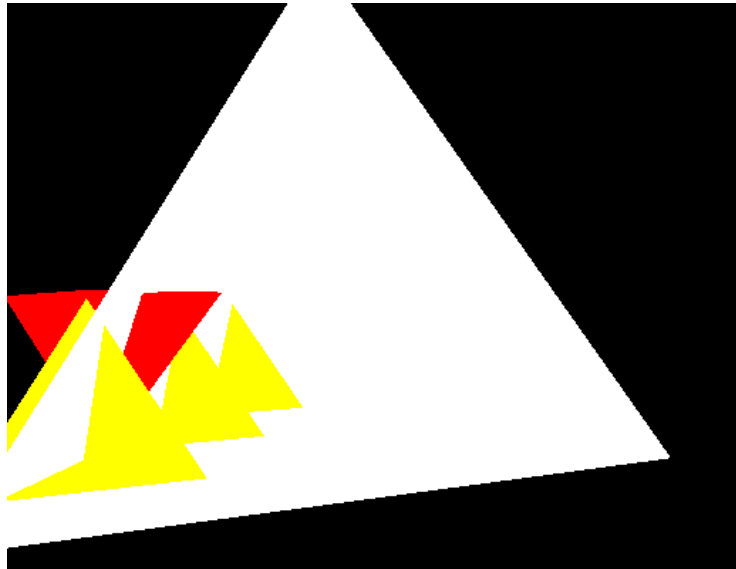

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Actividad errores/problemas:

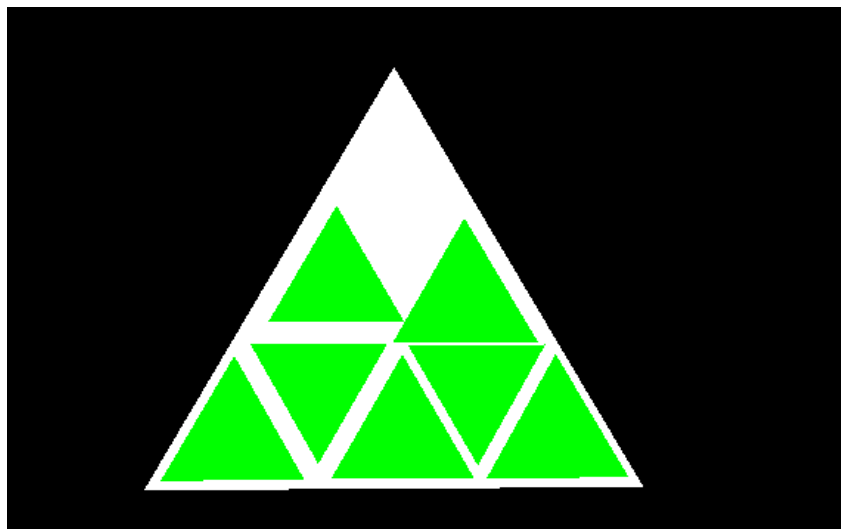
Por más que intente emplear pirámides estas no lograban acomodarse de forma adecuada, cuando intente poner una de las dimensiones en 0.0f esta no podía rotar adecuadamente, tampoco me fue posible manejar adecuadamente la cámara para estar revisando los ejes y cuando intentaba ver el progreso las pirámides con sus 3 valores x, y, z generaban colisiones inesperadas, por esto mismo únicamente realice 2 caras y admito que no pude acabar la práctica de forma adecuada ya que por tiempo intente arreglar los problemas antes mencionados y esto provoco que no pudiera acabar en forma adecuada lo cual me resulta un fracaso doloroso.



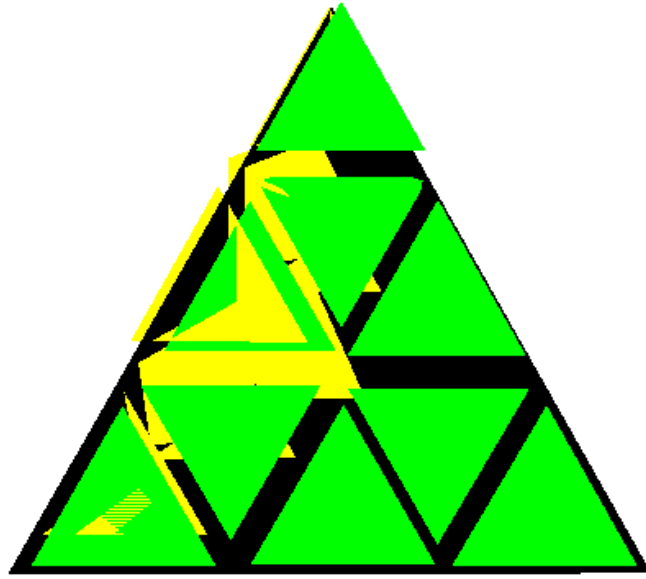
Error 1 rotación en pirámides con solo 2 escalamientos (no se giraba para poder sobre ponerse en la superficie)



Error 2 colisiones en vista lateral pese a tener correcta la cara frontal



Error 3 perspectiva (maneja la cámara en ocasiones hacia parecer que las piezas estaban correctamente acomodadas, pero girando un poco no resultaba correcto)



Error 4 superposición de 2 caras, solo visible desde el ángulo de la cara verde

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.

En esta ocasión el manejo de la cámara en conjunto con el uso de tantas instancias de pirámides para poder realizar el ejercicio resulto en un manejo caótico y lleno de errores el cual a pesar de dedicar múltiples intentos para poder solucionarlo me resulto en un ejercicio imposible de realizar de forma satisfactoria lo cual me frustró mucho y me hizo sentir mal dado que no era difícil el planteamiento del mismo sino su correcta implementación en el espacio tridimensional

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Referente a el ámbito teórico no tengo ninguna queja dentro del marco de la práctica, no obstante, el componente práctico me resulto en extremo complicado y muy difícil de manejar pues el ámbito del plano tridimensional resultaba más fácil en el apartado teórico que en la práctica.

c. Conclusión

De esta práctica puedo concluir que si bien los fundamentos teóricos para el modelado y estructura de elementos en entornos tridimensionales resulta accesible de comprender, llevarlo a la práctica comprende un desafío técnico ya que el manejo de cámaras, planos y rotaciones supone un constante conjunto de situaciones que requieren mayor cuidado para la realización de acciones que en el plano bidimensional resulta relativamente más sencillo.

Así mismo de esta práctica puedo comentar que por más esfuerzo que emplee para poder realizar la pyramix en una forma ordenada y adecuada no me fue posible completar dicho trabajo quedándome limitado a 2 caras las cuales no pude gestionar de la manera adecuada lo cual me frustra mucho y supone una invitación a repasar el apartado práctico de los temas vistos en teoría y en el laboratorio.

Bibliografía

- *LearnOpenGL - Advanced GLSL*. (n.d.). <https://learnopengl.com/Advanced-OpenGL/Advanced-GLSL>
- まるまるまる. (2020, June 3). i want to create 3D pyramid by OpenGL. Stack Overflow. <https://stackoverflow.com/questions/62164912/i-want-to-create-3d-pyramid-by-opengl>
- LearnOpenGL - Coordinate Systems. (n.d.). <https://learnopengl.com/Getting-started/Coordinate-Systems>
- An Introduction to `glmnet`. (n.d.). <https://glmnet.stanford.edu/articles/glmnet.html>
- GeeksforGeeks. (s. f.). Understanding "extern" keyword in C. <https://www.geeksforgeeks.org/understanding-extern-keyword-in-c/>